

Where Presence stands, and what comes next

A job-search companion people run themselves — status of the build as of 12 September 2026, with the work still ahead.

Prepared for Denis Baldakov (product owner) · internal, shareable with collaborators · companion documents: Values (2), Student track (3)

1. In one paragraph

Presence is the productisation of a personal job-hunt automation that has run in production since 13 August 2026 (six scheduled agents, seven job-source connector types, a 612-row application tracker, a Telegram delivery loop, and — since 12 September — automatic reading of application-status emails). The product keeps what made that system work and strips the rest: **one source, two agents, one messenger**, a small tracker core, everything essential free forever, and a charter enforced in code. Four of eight workstreams are complete; the runtime has been proven live against a frontier API and a free local model; the connector-to-tracker path is proven end to end. The next work is delivery (Telegram), then the agent packs that turn plumbing into judgment.

2. What has been decided

Decision	Outcome
What it is	An open, local-first operating system for the job hunt (and the study years before it): AI agents + a tracker with conventions + training tools + messenger delivery, run on the user's own machine and keys. The "Home Assistant model", aimed at careers.
v1 shape (canonical spec: <code>Presence_Architecture.pdf</code>)	Exactly one source (LinkedIn, library-based search), two agents (Scout, Brief), SQLite tracker, Telegram, BYO model key (Anthropic / OpenAI / Ollama — the zero-cost path), Docker, onboarding as a conversation (CV upload → parsed draft → five questions → confirm).
Business model	Everything essential is free, forever — no tiers, no subscriptions. Funding by voluntary support (Patreon / GitHub Sponsors / Open Collective). Paid may only ever be "extra, not essential". Training sims and drills are editable packs in the free core.
Charter (enforced by tests)	A human clicks every Submit · no invented facts · data stays on the user's machine · spending is capped daily.
Licensing	AGPL-3.0 core; MIT for pack and SDK formats so community content stays frictionless.
Repository	<code>github.com/Sofa3xpert/presence</code> , private for now, single contributor, CI (lint · tests · Docker build) green on every push. Commit history is minimal and product-focused.
Tracker schema	Small fixed engine core plus an uninterpreted <code>extra</code> field per job; new features become new interpreters, not migrations.

3. Work completed (11–12 September 2026)

Step	Delivered	Contents
W0	Foundations	Monorepo (<code>src/presence/{core,agents,connectors,adapters,tracker,packs}</code>), licences, README with the free-forever story, CHARTER.md, CONTRIBUTING, issue templates, CI, Dockerfile, uv toolchain. The charter is guarded by a test that fails if any rule is removed.
W1	Core runtime	Provider-agnostic agent executor (a tool-use loop; two adapters cover Anthropic, OpenAI, Ollama and LM Studio); daily token budget with a notify-once hook; supervisor with grace windows, cooldowns, attempt caps, dependencies and catch-up after sleep; YAML config with human-worded errors (an unconfirmed profile is refused — charter rule 2 as code); local-only secret lookup; a live check command. M1 signed off live : Anthropic (claude-haiku-4-5) and Ollama (qwen3.5:9b) both drove the loop through a tool call to completion.
W2	Tracker & conventions engine	Stdlib SQLite, no ORM; versioned migrations; statuses <code>to_apply · applied · assessment · interview · interview_done · rejected · ignored</code> ; the timeline ("29 Aug → CCAT 1 Sep → rejected 8 Sep") is rendered from an events table, never stored; closed jobs accept only notes; cooldown registry; CSV export; online backup. Precision-first dedupe : same link key merges; identical title merges only when one side has no link; two different links are two postings. Validated read-only against the real 612-row sheet.
W3	Connectors	One connector interface with a registry; LinkedIn connector on python-jobsby with per-query error isolation; queries built from <code>search.yaml</code> (freshness window, blocklist, results cap); seen-postings store so a judged posting never resurfaces; live check ran a real search of 10 postings straight into a tracker.
Test suite: 43 tests, none touching the network. Tooling lesson recorded: an environment "doctor" script and test-path independence were added after a macOS venv quirk cost debugging time.		

4. What remains for v1

Step	Deliverable	Notes / definition of done
W4	Telegram adapter	Messenger interface with Telegram as first transport; pairing by token/QR; daily brief delivery; <code>/status /pause /redo</code> ; the budget-exhausted hook wired to a message.
W6	Onboarding & docs	CV upload → parsed draft → five questions → confirmation → first run inside 15 minutes; CLI fallback; docs site (quickstart, connector and pack authoring, the charter). Closes M3 — "it talks to you" and triggers the dogfood switch: the author moves his own hunt onto Presence.
W5	Agent packs + evals	De-personalised Scout and Brief prompts in a shared pack format; an eval harness of fixture jobs with expected keep/drop/tier decisions, run against Claude, GPT and a local model. Closes M2 — "Scout fills a tracker" (the plumbing beneath it is already proven) and gates M4 — "anyone can install it" .
W7	Hardening & beta	Recorded-fixture connector tests; full daily cycle in Docker; macOS / Linux / Windows-WSL; private beta of 5–10 users; two fix cycles. Closes M5 — v1.0 .

v1.0 definition of done (unchanged): fresh machine to working bot in under 15 minutes · three providers pass the eval suite · zero-cost path (Ollama) verified end to end · kill the container mid-run → clean recovery · token cap fires before overspend · tracker survives 30 days unattended · charter enforced in tests · secrets absent from logs · backup/restore tested · connector guide validated by an outside contributor · v0.9 → v1.0 migration on a real database · five beta users complete a week unassisted.

5. Backlog beyond v1 (recorded, not scheduled)

- **Email-alerts connector** — the customer's saved-search alert emails as a second source; the registry already has the slot.
- **Status-mail loop** — reading application outcomes from the mailbox (read-only scan → agent judgment → one guarded writer). Built and running in the personal pipeline since 12 September; ports almost as-is.
- **Training studio** — assessment drills, interview sims with an AI facilitator, daily problem sets; v1 ships only the pack format. Also the natural meeting point with the student track (Document 3).
- More sources (ATS public APIs, region watchlists), Sheets/Notion tracker backends, WhatsApp/Discord adapters, desktop app packaging, an optional hosted runner — all explicitly later.

6. Open questions and risks

- **Judgment on cheap models** — how much of Scout's keep/drop/tier quality survives on a local 9B model is unknown until the eval harness exists (W5). Decide by measurement, not opinion.
- **Board scraping is terms-of-service grey.** v1 accepts it as the pragmatic start (each user runs as themselves); the clean channels (email alerts, public ATS APIs) are the strategic direction.
- **Brief timing and volume** — one digest versus instant pings for top-tier finds; decide during W4/W6 with real use.
- **"Anywhere" versus "no service"** — a phone-only user cannot self-host; the hosted runner stays an optional convenience decided later.
- **Time** — the job hunt outranks product weeks; milestones are self-contained so pauses cost nothing but calendar.

7. Next actions

1. Brand foundations (logo, style, brand-book) — in parallel with build work, now that the three documents define what the brand must express.
2. W4 Telegram adapter — the next build session.
3. Student-track kickoff with the collaborator (Document 3).